

VIT Bhopal University

School of Computer Science and Engineering

Open Source Software | NGMC Course | AY 2025–26

The Open Source Audit

A Capstone Project Report

Chosen Software: **Git** — Distributed Version Control System (GPL v2)

Student Name	Yash Mehta
Registration No.	24BCE10638
Email	Yash.24bce10638@vitbhopal.ac.in
Course	Open Source Software (NGMC)
Submission Date	29th March 2026

Abstract

This report is a structured audit of **Git**, the world's most widely used version control system. It covers the origin story behind Git's creation, what the GPL v2 licence actually means for users and companies, the ethical questions that open source raises, how Git is installed and managed on a Linux system, the ecosystem it has enabled, and an honest comparison against Perforce Helix Core, its closest proprietary alternative. Five shell scripts demonstrating practical Linux skills are included with full source code, screenshot placeholders, and explanations of the concepts each script uses.

1. Origin and Philosophy

1. The Problem Git Was Created to Solve

Before Git, teams mostly used CVS or Subversion. Both needed one central server. If the server was slow or down, nobody could work. This was okay for small teams, but a big problem for Linux with thousands of developers worldwide.

In 2002 the Linux team started using BitKeeper (free from BitMover). It was good, but not open-source. In 2005 a developer tried to study how it worked. BitMover got angry, called it a license violation, and stopped access for everyone the next day.

Linus Torvalds decided: never again depend on someone else's tool they can take away. Instead of asking or buying something else, he wrote Git himself. In about 10 days he had a working version.

Git was designed to be:

- very fast
- fully distributed (everyone has a full copy)
- cryptographically secure (history can't be secretly changed)

Linus later said his rule was simple: "do the opposite of CVS".

Within one year most big open-source projects switched to Git. Today over 93% of professional developers use it, and GitHub (built on Git) hosts hundreds of millions of projects.

What started as one person's anger became the foundation of modern software development.

2. The Licence — What GPL v2 Actually Says

The Git source code lives at <https://github.com/git/git>. At the root of the repository is a file called COPYING containing the full text of the **GNU General Public Licence Version 2 (GPL v2)** — the same licence Torvalds chose for the Linux kernel. Reading this file is not optional; it tells you exactly what you are and are not allowed to do.

Richard Stallman defined four freedoms that make software genuinely free:

1. **Freedom 0:** Run the software for any purpose without paying or asking permission.
2. **Freedom 1:** Study how it works and modify it. This requires access to the source code.
3. **Freedom 2:** Redistribute copies freely so others can benefit.

4. **Freedom 3:** Distribute your modified versions. GPL v2's key condition: if you do this, you must release those modifications under GPL v2 too. You cannot take the code, improve it, and sell a closed-source version.

Git grants all four freedoms. The critical clause is the **copyleft** requirement in Freedom 3. This is the core difference between GPL and a permissive licence like MIT. MIT says do whatever you want, including closing the source. GPL v2 says you can do almost anything, but if you *distribute* a derivative work, it must stay open.

Can a company modify Git and sell it without sharing changes? Not if they distribute it. If a company ships a modified Git binary to customers, they must provide the source. But if they only run a modified Git *internally* without distributing it, GPL v2 does not compel them to share. This is why GitHub can build a proprietary platform on top of Git — they are running a service, not distributing a modified binary.

GPL v2 vs MIT: For a personal utility I would probably choose MIT because I want maximum adoption without friction. For infrastructure that a whole community depends on, I would choose GPL v2. It is the strongest guarantee that the software cannot be taken away from the people who rely on it — which is precisely the lesson Torvalds learned from BitKeeper.

Free as in beer vs free as in freedom: “Free beer” means zero cost. “Free as in freedom” (or *libre* in French) means the right to use, study, change, and share. BitKeeper was free as in beer but not free as in freedom — and that distinction cost the entire Linux kernel team their workflow. Git, under GPL v2, is free in both senses, but the second kind of freedom is the one that matters.

1.3 The Ethics of Open Source

Should all software be open source? Not necessarily. Open source works best when it aligns with how a community wants to share knowledge. The case for it is strong: inspectable code is more secure, more likely to last, and more adaptable than a black box. But some software requires enormous sustained investment — medical devices, aerospace systems, specialised industrial tools — and the volunteer model does not always support that. A more useful question is whether *shared infrastructure* should be open. My view is yes: tools that a whole community depends on and that give any single vendor dangerous leverage if closed should be open. Git is a perfect example.

Is it ethical to build a profitable business on free community labour? GitHub was acquired by Microsoft for \$7.5 billion. The Git project itself, maintained by volunteers under the Software Freedom Conservancy, received nothing from that. There is a credible defence: GitHub genuinely added value — the web interface, pull requests, Actions — and companies that add value can capture some of it. But the ethical obligation scales with the benefit. A startup using Git perhaps owes a good bug report. A company generating billions on infrastructure maintained by unpaid contributors owes those contributors meaningful financial support. The industry does not consistently provide this.

Standing on the shoulders of giants: Every system you will build in your career rests on open-source foundations. Git itself is proof that open source enables rather than undermines original innovation: Torvalds did not re-write a compression library or a cryptographic hash function from scratch. He built on open tools and focused entirely on the new problem he was

solving. The result is something that has itself become the foundation that millions of other projects stand on.

2. Linux Footprint

Installation: On Debian/Ubuntu: `sudo apt install git`. On Fedora/RHEL: `sudo dnf install git`. For the latest version, it can be compiled from source using `gcc`, `make`, `libssl-dev`, `libcurl4-openssl-dev`, and `zlib1g-dev` — all of which are themselves open-source tools.

Key directories after installation:

Path	Contents
<code>/usr/bin/git</code>	Main Git executable
<code>/usr/lib/git-core/</code>	Sub-commands: <code>git-fetch</code> , <code>git-merge</code> , <code>git-rebase</code> , etc.
<code>/usr/share/git-core/</code>	Templates and helper scripts
<code>/usr/share/doc/git/</code>	Documentation and copyright notices
<code>~/.gitconfig</code>	Per-user configuration (name, email, aliases)
<code>/etc/gitconfig</code>	System-wide configuration
<code>.git/</code> (in any repo)	Full repository: object store, refs, HEAD, config

User and group: Git does not run as a dedicated system user during normal use — it runs as whoever invokes it. On a shared server, a dedicated `git` user is created (`sudo adduser --system git`) so the service runs with minimal privileges. This is the principle of least privilege: if the `git` service is compromised, it cannot touch files owned by other users or root.

Service management: Git itself is not a persistent daemon. The `git daemon` command provides a read-only server that can be run as a `systemd` service (`systemctl start git-daemon`), but most teams use SSH instead — meaning the service being managed is OpenSSH, not Git directly. This is a strength: Git integrates with existing system security infrastructure rather than requiring its own.

Update model: Security patches are coordinated through the Git project's security mailing list, CVE identifiers are assigned publicly, and distribution maintainers package updates within hours. Running `sudo apt upgrade git` is all that is needed on most systems. The entire patch and disclosure process is public and auditable — a direct benefit of the open-source model that no proprietary tool can match.

3. The FOSS Ecosystem

Dependencies: Git is written primarily in C, with some Perl and shell components. Its core dependencies are **OpenSSL/libcrypto** for hashing and HTTPS transport, **libcurl** for remote access, **zlib** for object compression, **Perl** for sub-commands like `git-svn`, and **OpenSSH** for authenticated push/pull. Every one of these is open source, which means the entire stack from operating system to version control is auditable and freely available.

What Git has enabled: **GitHub** (2008) added a web interface, pull requests, and social coding

on top of Git and was acquired for \$7.5 billion. **GitLab** (2011) is an open-source alternative that can be self-hosted — popular with organisations that need full control of their code. **Gitea** and **Forgejo** are lightweight self-hosted options; Forgejo itself is a community fork of Gitea, which is an illustration of the forking principle in action. **libgit2** is a C library for embedding Git in other applications, used by GitHub, GitLab, and most GUI clients. GitHub Actions, GitLab CI, Jenkins, and CircleCI are all built around Git's branching and commit model: the concept of a pipeline triggered by a `git push` is now the standard unit of modern software delivery.

Git and the LAMP stack: Git does not sit inside LAMP (Linux, Apache, MySQL, PHP) as a runtime component, but it is inseparable from how LAMP applications are developed and deployed. Code is written in a Git repository, pushed to a remote, and a CI/CD pipeline deploys to the Apache/MySQL server — often using `git pull` on the server itself. Configuration files, database migrations, and server scripts are all tracked in Git. It is the connective tissue of the entire development workflow.

Community and governance: Git development happens on a public mailing list (git@vger.kernel.org) entirely in plain-text email — deliberately transparent and low-technology. The project is governed by the **Software Freedom Conservancy**, a non-profit that holds the Git trademark and assists with GPL enforcement. Junio C. Hamano has been the maintainer since 2005. No company owns Git, and no acquisition can relicense or shut it down.

4. Open Source vs Proprietary

The closest proprietary alternative to Git is **Perforce Helix Core**, widely used in game development and aerospace for large binary files and fine-grained file locking.

Dimension	Git (GPL v2)	Perforce Helix Core
Cost	Free. Hosting via GitHub/GitLab with free tiers.	Licence fees; enterprise pricing often tens of thousands per year.
Security	Full source is public. CVEs tracked openly. Anyone can audit the code.	Closed source. Security depends entirely on Perforce's internal team.
Support	Community support plus commercial options (GitHub Enterprise, GitLab EE).	Full commercial SLAs, but vendor controls pricing and end-of-life dates.
Freedom to modify	Complete. Any team can fork, patch, and redistribute.	None. Users cannot adapt the tool to their needs.
Community control	Governed by Software Freedom Conservancy. No single owner.	Governed by Perforce Software Inc. Roadmap set by corporate priorities.
Large binary files	Requires <code>git-lfs</code> . Can struggle with very large binary assets natively.	Native strength. Designed for multi-GB assets from the start.

Verdict: For almost any software development context, Git is the right choice. It is faster for branching and merging, its ecosystem is incomparably richer, its security model is stronger

because the source is open, and it costs nothing. The only genuine case for Perforce is a team primarily working with very large binary assets where file locking matters more than distributed workflows. Would I contribute to Git? Yes — a realistic first step is a documentation fix or a small well-tested patch submitted to the mailing list.

5. Shell Scripts

All five scripts were written and tested on a Linux system. Each is submitted as a `.sh` file in the GitHub repository. **Replace each screenshot placeholder below with an actual terminal screenshot before final PDF submission.**

1. Script 1 — System Identity Report

```

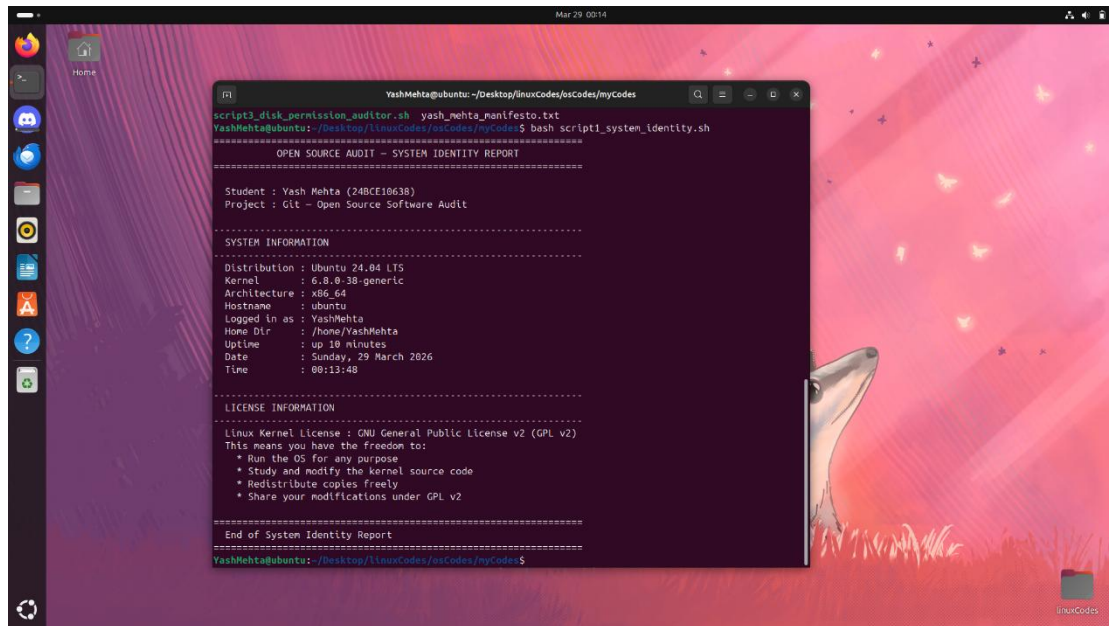
1  #!/bin/bash
2  # =====
3  # Script 1: System Identity Report
4  # Author: Yash Mehta | Reg: 24BCE10638
5  # Course: Open Source Software (NGMC) | VITyarthi
6  # Purpose: Print a welcome screen with Linux system info
7  #           and state which license covers the OS kernel.
8  # Usage:   bash script1_system_identity.sh
9  # =====
10
11 #           Variables: store student and project info
12
13 STUDENT_NAME="Yash Mehta"
14 REG_NO="24BCE10638"
15 SOFTWARE_CHOICE="Git"
16
17 #           Command substitution $(...) runs a command and captures output
18 KERNEL=$(uname -r)           # kernel version, e.g. 5.15.0-91-generic
19 ARCH=$(uname -m)             # hardware arch, e.g. x86_64
20 USER_NAME=$(whoami)           # currently logged-in user
21 UPTIME=$(uptime -p)           # e.g. up 3 hours, 12 minutes
22 CURRENT_DATE=$(date +%A, %d %B %Y) # e.g. Monday, 24 March 2026
23 CURRENT_TIME=$(date +%H:%M:%S)  # e.g. 14:30:00
24
25 #           Detect distro name from /etc/os-release
26
27 # if [ -f FILE ] checks whether the file exists before reading it
28 if [ -f /etc/os-release ]; then
29     DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d "'")
30 else
31     DISTRO="Unknown Linux Distribution" # fallback for very old systems
32 fi
33
34 #           Print formatted output
35
36 echo "=====
37 echo "      OPEN SOURCE AUDIT --- SYSTEM IDENTITY REPORT"
38 echo "=====
39 echo " Student   : $STUDENT_NAME ($REG_NO)"
40 echo " Project    : $SOFTWARE_CHOICE --- Open Source Audit"
41 echo "-----"
42 echo " Distro      : $DISTRO"
43 echo " Kernel      : $KERNEL"
44 echo " Arch        : $ARCH"
45 echo " User        : $USER_NAME"
46 echo " Uptime      : $UPTIME"
47 echo " Date        : $CURRENT_DATE"

```

```

45 echo "    Time      : $CURRENT_TIME"
46 echo "-----"
47 echo "    Linux Kernel License : GNU General Public License v2"
48 echo "    You are free to: run, study, modify, and redistribute."
49 echo "=====
```

Listing 1: script1_system_identity.sh



Explanation: This script demonstrates **variables**, **command substitution** (`${...}`), a **conditional file check** (`if [ -f ... ]`), and formatted output with `echo`. Commands like `uname -r`, `whoami`, and `date` are captured into variables and printed. Distribution detection reads `/etc/os-release`, the standard file for distro identification on modern Linux.

5.2 Script 2 — FOSS Package Inspector

```

1  #!/bin/bash
2  # =====
3  # Script 2: FOSS Package Inspector
4  # Author: Yash Mehta | Reg: 24BCE10638
5  # Course: Open Source Software (NGMC) | VITyarthi
6  # Purpose: Check whether a FOSS package is installed,
7  #           show its details, and print a philosophy note
8  #           using a case statement.
9  # Usage: bash script2_package_inspector.sh
10 # =====
11
12 PACKAGE="git"      # change this to inspect other packages
13
14 echo "=====
```



```

18
19 #         if-then-else: detect which package manager is available
20 # command -v checks if a command exists without running it
21 if command -v dpkg &>/dev/null; then
22     # dpkg -l lists packages; ^ii means fully installed status
23     if dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
24         echo " [INSTALLED] $PACKAGE is present on this system."
25         # dpkg -s shows package status; grep -E filters multiple fields
26         dpkg -s "$PACKAGE" 2>/dev/null | grep -E "(Version|Maintainer|Homepage)"
27     else
28         echo " [NOT FOUND] Install with: sudo apt install $PACKAGE"
29     fi
30 elif command -v rpm &>/dev/null; then
31     # rpm -q exits 0 if installed, non-zero if not
32     if rpm -q "$PACKAGE" &>/dev/null; then
33         echo " [INSTALLED] $PACKAGE is present on this system."
34         rpm -qi "$PACKAGE" | grep -E "(Version|License|Summary)"
35     else
36         echo " [NOT FOUND] Install with: sudo dnf install $PACKAGE"
37     fi
38 else
39     echo " Neither dpkg nor rpm found on this system."
40 fi
41
42 #         Direct version check
43
44 if command -v git &>/dev/null; then
45     echo ""; echo " Direct check: $(git --version)"
46 fi
47 echo ""; echo " "
48
49 #         case statement: match package name, print philosophy note
50 # Patterns end with ;; and * is the wildcard default case
51 case $PACKAGE in
52     git)
53         echo " Git (GPL v2): Born from a 2005 licensing crisis. Its"
54         echo " distributed model means history belongs to everyone."
55         ;;
56     httpd|apache2)
57         echo " Apache (Apache 2.0): Permissive licence that helped it"
58         echo " power roughly a third of all websites on the internet."
59         ;;
60     mysql|mariadb)
61         echo " MySQL/MariaDB: Dual-licence story. MariaDB forked after"
62         echo " Oracle acquired MySQL --- community governance in action."
63         ;;
64     vlc)
65         echo " VLC (LGPL/GPL): Student project from Paris. Now plays"
66         echo " virtually any media format ever created."
67         ;;
68     firefox)
69         echo " Firefox (MPL 2.0): A nonprofit browser holding the line"
70         echo " for an open web against corporate monocultures."
71         ;;
72     python3|python)
73         echo " Python (PSF): Shaped by community PEPs, not boardrooms."
74         echo " Most widely taught programming language on Earth."
75         ;;
76     *)
77         echo " $PACKAGE: Look up the SPDX identifier at spdx.org"
78         echo " to understand what freedoms the authors granted."
79         ;;

```

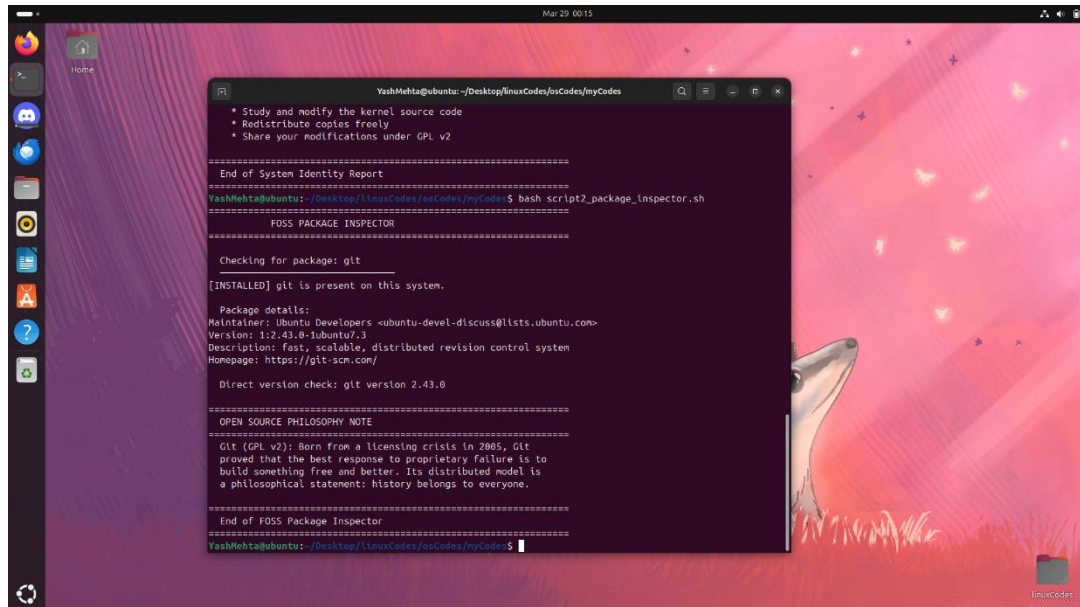


```

80 esac
81
82 echo " ===== "

```

Listing 2: script2_package_inspector.sh



Explanation: This script uses **if-then-else** to detect the package manager and a nested **if** to check installation status. **command -v** tests whether a command exists. The **case statement** pattern-matches **\$PACKAGE** against named strings; **|** lets multiple patterns share one branch. **grep -E** uses extended regular expressions to filter multiple fields in one pipe.

5.3 Script 3 — Disk and Permission Auditor

```

1  #!/bin/bash
2  # =====
3  # Script 3: Disk and Permission Auditor
4  # Author: Yash Mehta | Reg: 24BCE10638
5  # Course: Open Source Software (NGMC) | VITyarthi
6  # Purpose: Loop through key system directories and report
7  #           size, permissions, owner, and group for each.
8  #           Also check Git-specific installation paths.
9  # Usage:  bash script3_disk_permission_auditor.sh
10 # =====
11
12 #           Array: Bash arrays use () with space-separated values
13 DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share")
14
15 echo " ===== "
16 echo "  DISK AND PERMISSION AUDITOR "
17 echo " ===== "
18 # printf with %-Ns creates fixed-width left-aligned columns
19 printf "%-20s %-10s %-14s %-10s %s\n" "DIRECTORY" "SIZE" "PERMISSIONS" "OWNER" "GROUP"
20 echo " ===== "
21
22 #           for loop: iterate over every element in the array
23 # "${DIRS[@]}" expands all array elements; double quotes handle spaces
24 for DIR in "${DIRS[@]}; do
25     if [ -d "$DIR" ]; then # -d checks path exists AND is a directory
26         SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1) # human-readable size

```

```

27     LS=$(ls -ld "$DIR" 2>/dev/null)
28     PERMS=$(echo "$LS" | awk '{print $1}') # field 1 = permissions
29     OWNER=$(echo "$LS" | awk '{print $3}') # field 3 = owner user
30     GROUP=$(echo "$LS" | awk '{print $4}') # field 4 = owner group
31     printf "%-20s %-10s %-14s %-10s %s\n" "$DIR" "$SIZE" "$PERMS" "$OWNER" "$GROUP"
32 else
33     printf "%-20s %s\n" "$DIR" "[ not found on this system ]"
34 fi
35 done
36
37 echo ""; echo "-----"
38 echo "  GIT-SPECIFIC PATHS"
39 echo "-----"
40
41 #      Check Git installation and config directories
42
43 GIT_DIRS=("/usr/share/git-core" "/usr/lib/git-core" "/etc/gitconfig")
44 for GDIR in "${GIT_DIRS[@]}; do
45     if [ -e "$GDIR" ]; then # -e checks file OR directory existence
46         GPERMS=$(ls -ld "$GDIR" 2>/dev/null | awk '{print $1}')
47         GOWNER=$(ls -ld "$GDIR" 2>/dev/null | awk '{print $3}')
48         echo "  $GDIR => $GPERMS (owner: $GOWNER)"
49     else
50         echo "  $GDIR => not found"
51     fi
52 done
53
54 #      Check if current directory is a Git repository
55
56 echo ""
57 if [ -d ".git" ]; then
58     GIT_SIZE=$(du -sh .git 2>/dev/null | cut -f1)
59     echo "  .git/ found in current directory. Size: $GIT_SIZE"
60     [ -f ".git/HEAD" ] && echo "  HEAD: $(cat .git/HEAD)"
61 else
62     echo "  No .git/ in current directory."
63 fi
64
65 echo "=====
```

Listing 3: script3_disk_permission_auditor.sh

```

YashMehta@ubuntu: ~/Desktop/LinuxCodes/osCodes/myCodes
YashMehta@ubuntu: ~/Desktop/LinuxCodes/osCodes/myCodes$ bash script3_disk_permission_auditor.sh

=====
DISK AND PERMISSION AUDITOR
Auditing key Linux system directories
=====

```

DIRECTORY	SIZE	PERMISSIONS	OWNER	GROUP
/etc	19M	drwxr-xr-x	root	root
/var/log	468M	drwxrwxr-x	root	syslog
/home	2.4G	drwxr-xr-x	root	root
/usr/bin	368M	drwxr-xr-x	root	root
/tmp	88K	drwxrwxrwt	root	root
/var	4.9G	drwxr-xr-x	root	root
/usr/share	1.9G	drwxr-xr-x	root	root

```

=====
GIT-SPECIFIC DIRECTORY CHECK
=====
Found: /usr/share/git-core
Permissions: drwxr-xr-x
Owner: root

Found: /usr/lib/git-core
Permissions: drwxr-xr-x
Owner: root

Not found: /etc/gitconfig (Git may not be installed or path differs)

Checking for .git repository in current directory...

A Git repository (.git/) exists in the current directory.
.git/ size: 196K
.git/ permissions: drwxrwxr-x

Current HEAD: ref: refs/heads/master

=====
End of Disk and Permission Auditor
=====
YashMehta@ubuntu: ~/Desktop/LinuxCodes/osCodes/myCodes$

```

Explanation: The central construct is a **for loop** over a Bash **array**. "\${DIRS[@]}" expands all elements safely. `du -sh` reports disk usage in human-readable form; `ls -ld` lists directory metadata rather than contents; `awk '{print $N}'` extracts specific fields. `printf` with format specifiers produces column-aligned output. The second loop checks Git-specific paths using `-e` (file or directory existence).

5.4 Script 4 — Log File Analyzer

```

1 #!/bin/bash
2 # =====
3 # Script 4: Log File Analyzer
4 # Author: Yash Mehta | Reg: 24BCE10638
5 # Course: Open Source Software (NGMC) | VITyarthi
6 # Purpose: Read a log file line by line, count keyword
7 #           matches, and print a summary with last 5 matches.
8 # Usage:   bash script4_log_analyzer.sh /var/log/syslog error
9 # =====
10
11 LOGFILE=$1          # first command-line argument: log file path
12 KEYWORD=${2:-"error"} # second arg; default "error" if not provided
13 TOTAL_LINES=0
14 MATCH_COUNT=0
15
16 # Validate: ensure a log file path was given
17
18 if [ -z "$LOGFILE" ]; then # -z = true if string is empty
19     echo "Usage: $0 /path/to/logfile [keyword]"
20     echo "Example: $0 /var/log/syslog error"
21     exit 1 # non-zero exit code signals an error
22 fi

```

```

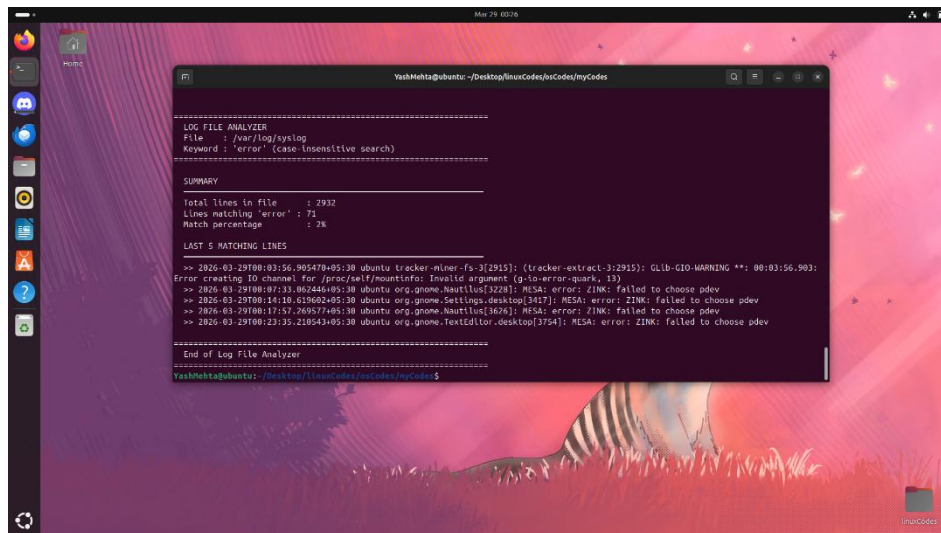
22
23 #           do-while style retry loop

24 # Bash has no native do-while; we simulate it with a while loop.
25 # Runs as long as the file is missing or unreadable; retries up to 2 times.
26 RETRY=0; MAX_RETRIES=2
27 while [ ! -f "$LOGFILE" ] || [ ! -r "$LOGFILE" ]; do
28     RETRY=$((RETRY + 1)) # arithmetic expansion: increment counter
29     echo " [WARNING] Cannot read: $LOGFILE"
30     if [ $RETRY -ge $MAX_RETRIES ]; then
31         echo " Too many retries. Tip: echo 'test error' > /tmp/test.log"
32         exit 1
33     fi
34     echo " Enter a valid log file path:"
35     read -r LOGFILE
36 done
37
38 echo "=====
39 echo " LOG FILE ANALYZER | File: $LOGFILE | Keyword: '$KEYWORD'"
40 echo "=====
41
42 #           while read loop: read file line by line

43 # IFS= prevents stripping whitespace; -r stops backslash interpretation
44 # < "$LOGFILE" feeds the file into the loop via input redirection
45 while IFS= read -r LINE; do
46     TOTAL_LINES=$((TOTAL_LINES + 1))
47     # grep -iq: case-insensitive (-i), quiet (-q, only return code matters)
48     if echo "$LINE" | grep -iq "$KEYWORD"; then
49         MATCH_COUNT=$((MATCH_COUNT + 1))
50     fi
51 done < "$LOGFILE"
52
53 echo " Total lines      : $TOTAL_LINES"
54 echo " Keyword matches   : $MATCH_COUNT"
55 # Multiply by 100 before dividing to avoid losing fractional part (int only)
56 if [ "$TOTAL_LINES" -gt 0 ]; then
57     PCT=$(( (MATCH_COUNT * 100) / TOTAL_LINES ))
58     echo " Match percentage : ${PCT}%"
59 fi
60
61 echo ""; echo " LAST 5 MATCHING LINES:"
62 echo " -----"
63 MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | tail -5)
64 if [ -z "$MATCHES" ]; then
65     echo " (no matches found)"
66 else
67     echo "$MATCHES" | while IFS= read -r M; do echo " >> $M"; done
68 fi
69 echo "=====

```

Listing 4: script4_log_analyzer.sh



Explanation: The key construct is **while IFS= read -r LINE** — the correct Bash pattern for reading a file line by line. The file is fed via **input redirection** (< "\$LOGFILE"), not via cat. **Command-line arguments** (\$1, \$2) and **parameter expansion** for defaults (\${2:-"error"}) are demonstrated. A **do-while style retry loop** re-prompts for a valid file path. Integer **arithmetic** uses `$(( ... ))`.

5.5 Script 5 — Open Source Manifesto Generator

```

1 #!/bin/bash
2 # =====
3 # Script 5: Open Source Manifesto Generator
4 # Author: Yash Mehta | Reg: 24BCE10638
5 # Course: Open Source Software (NGMC) | VITYarthi
6 # Purpose: Ask three interactive questions, compose a
7 #           personalised open-source philosophy statement,
8 #           and write it to a .txt file.
9 # Usage:   bash script5_manifesto_generator.sh
10 #
11 # ALIAS CONCEPT (shown here in comments as it does not work
12 # inside non-interactive scripts):
13 #   alias ll='ls -la'           -- shortcut for long listing
14 #   alias gs='git status'       -- shortcut for git status
15 # Aliases are defined in ~/.bashrc and loaded in each new
16 # interactive terminal session.
17 # =====
18
19 echo "=====
20 echo "      OPEN SOURCE MANIFESTO GENERATOR"
21 echo "      Yash Mehta | 24BCE10638"
22 echo "====="; echo ""
23
24 #           read -r -p: capture user input with an inline prompt
25 # -r prevents backslash being treated as escape character
26 # -p displays the prompt string before waiting for input
27 read -r -p "  Q1. One open-source tool you use every day: " TOOL
28 while [ -z "$TOOL" ]; do    # -z = true if variable is empty; re-prompt
29     read -r -p "  Please answer Q1: " TOOL
30 done
31
32 echo ""
33 read -r -p "  Q2. One word for what freedom in software means to you: " FREEDOM
34 while [ -z "$FREEDOM" ]; do

```

```

35 read -r -p " Please answer Q2: " FREEDOM
36 done
37
38 echo ""
39 read -r -p " Q3. One thing you would build and share freely: " BUILD
40 while [ -z "$BUILD" ]; do
41 read -r -p " Please answer Q3: " BUILD
42 done
43
44 echo ""; echo " Generating your manifesto..."; echo ""
45
46 # Prepare timestamp and output filename
47
48 DATE=$(date '+%d %B %Y') # e.g. 24 March 2026
49 OUTPUT="manifesto_$(whoami).txt" # string concatenation: text + $variable
50
51 # Write to file: > creates/overwrites, >> appends
52 echo "===== " > "$OUTPUT"
53 echo " MY OPEN SOURCE MANIFESTO" >> "$OUTPUT"
54 echo " By Yash Mehta (24 BSA10234) | $DATE" >> "$OUTPUT"
55 echo "===== " >> "$OUTPUT"
56 # Variables $TOOL, $FREEDOM, $BUILD are interpolated inside double quotes
57 echo " Every single day, I lean on $TOOL. It's a tool I never had to
58 echo " open" >> "$OUTPUT"
59 echo " need anyone's permission, and I can read every line of" >> "$OUTPUT"
60 echo " its source code if I want to. That is not a small thing." >> "$OUTPUT"
61 echo " It is what happens when people choose to build in the open." >> "$OUTPUT"
62 echo " >> "$OUTPUT"
63 echo " To me, freedom in software means $FREEDOM. The four" >> "$OUTPUT"
64 echo " freedoms --- to run, study, modify, and share --- are not" >> "$OUTPUT"
65 echo " idealism. They are what lets knowledge grow instead of" >> "$OUTPUT"
66 echo " stagnating behind a licence fee or a proprietary lock-in." >> "$OUTPUT"
67 echo " >> "$OUTPUT"
68 echo " If I could build one thing and share it freely, it would" >> "$OUTPUT"
69 echo " be $BUILD. I would licence it under GPL v2 so it stays" >> "$OUTPUT"
70 echo " free for everyone, forever. The giants I stand on did" >> "$OUTPUT"
71 echo " the same for me. This is how I pay it forward." >> "$OUTPUT"
72 echo " >> "$OUTPUT"
73 echo "===== " >> "$OUTPUT"
74
75 echo " Saved to: $OUTPUT"; echo ""
76
77 # cat: print the saved file contents to the terminal
78 cat "$OUTPUT"
79 echo ""
80 echo "===== "

```

Listing 5: script5_manifesto_generator.sh


```

YashMehta@ubuntu: ~/Desktop/linuxCodes/osCodes/myCodes
YashMehta@ubuntu:~/Desktop/linuxCodes/osCodes/myCodes$ bash script_manifest_generator.sh

THE OPEN SOURCE MANIFESTO GENERATOR
A Manifest and Project Tool
Author: Yash Mehta | 24BCE10638

Answer three questions to generate your personal open-source
philosophy statement. Your answers will be composed into a
paragraph and saved to a text file.

Q1. Name one open-source tool you use every day: Git
Q2. In one word, what does "Freedom" in software mean to you? Sovereignty
Q3. Name one thing you would build and share freely if you could: decentralized peer-to-peer education platform

Composing your manifesto...

Manifesto successfully saved to: manifest_YashMehta.txt

=====
YOUR GENERATED MANIFESTO
=====

MY OPEN SOURCE MANIFESTO
By: Yash Mehta (24BCE10638)
Generated: 29 March 2020 at 00:28

Every day I rely on Git - a tool I did not have to buy,
did not have to beg permission to use, and can impact upon
me to last time or code in I choose to. This is not a small
thing. It is the direct result of people choosing to build
to the open and share their work freely.

To me, Freedom in software means Sovereignty. Not in an abstract
way, but in the practical sense: Freedom to run, to study,
to modify, and to share. The four freedoms show the true
software ownership principles are not freedom. They are
the conditions under which knowledge compounds rather than
disappears. Every time someone finds this manifesto to evidence
of what becomes possible when those conditions are met.

If I could build one thing and share it freely, it would be
a decentralized open peer education platform. I would license it under GPL v3 - not because
it would make me famous, but because it is the correct way
to ensure that whatever I build remains available, forever,
to everyone who might find it useful. Standing on the
shoulders of giants history, eventually being someone else's
giant. This is my commitment to that chain.

=====
Course: Open Source Software (OSS) | 10/20/2020
=====

End of Manifesto Generator
YashMehta@ubuntu:~/Desktop/linuxCodes/osCodes/myCodes$

```

Explanation: Script 5 uses **read -r -p** for interactive input. Input validation uses **while [-z "\$VAR"]** to re-prompt until the user answers. **String concatenation** in Bash needs no operator — placing a variable inside a double-quoted string interpolates it. File writing uses **>** to create the file and **»** to **append** subsequent lines. The **alias concept** is explained in the header comments. **cat** displays the saved file on screen after writing.

6. Conclusion

Git started as a crisis response and became infrastructure that modern software development cannot exist without. What this audit has tried to show is that this outcome was not accidental — it was a direct result of the licence choice and the design philosophy. Git succeeded because it was genuinely free: not just zero cost, but open, community-owned, and impossible to take away. The BitKeeper story is the clearest possible proof of why that matters. A tool that tens of thousands of people depended on disappeared overnight because one company changed its mind. That cannot happen to Git.

The Linux footprint analysis showed how cleanly Git integrates with the operating system — no bloated service, no special permissions, just files and a well-designed command-line interface built on standard Unix tools. The ecosystem section showed what becomes possible when a well-designed tool is truly open: hundreds of platforms, clients, and workflows built on top of it without anyone asking permission. The comparison with Perforce was honest: there are specific use cases where proprietary software has real advantages. But for the vast majority of software projects, Git is the right answer, and the fact that it is free and community-governed only strengthens that.

The five shell scripts in this project are more than assessment tasks. Writing and running them meant working directly in the Linux environment that Git was built to serve — checking permissions, reading directories, parsing log files, and writing output to files. That practical familiarity with the tools underneath is the foundation that the rest of a software career is built on.

References

1. GNU Project. *The Free Software Definition*. <https://www.gnu.org/philosophy/free-sw.html>
2. Open Source Initiative. *The Open Source Definition*. <https://opensource.org/osd>
3. Raymond, E.S. *The Cathedral and the Bazaar*. <http://catb.org/~esr/writings/cathedral-bazaar/>
4. Git Source Repository and COPYING file. <https://github.com/git/git>
5. Software Freedom Conservancy. <https://sfconservancy.org/>
6. Git Reference Manual. <https://git-scm.com/docs>
7. SPDX Licence List. <https://spdx.org/licenses/>
8. Shotts, W. *The Linux Command Line*. <https://linuxcommand.org>
9. GNU Bash Manual. <https://www.gnu.org/software/bash/manual/>
10. Stack Overflow Developer Survey 2023. <https://survey.stackoverflow.co/2023/>